Plateforme de Support IA

Chatbot Intelligent Multi-Applications

Présenté par : Fatou FALL

Master 2 : Informatique - Génie Logiciel

Université : Iba Der Thiam de Thiès

Année académique : 2025 – 2026

Date : Avril 2026

Table des matières

1 Introduction **3**
1.1 Contexte du projet . . . 3
1.2 Objectif du MVP . . . 3
1.3 Technologies utilisées . . . 3

2 Architecture de la solution **4**
2.1 Architecture globale . . . 4
2.2 Composants principaux . . . 4

3 Interface Admin **5**
3.1 Présentation . . . 5
3.2 Palette de couleurs . . . 5
3.3 Interface Admin - Captures d'écran . . . 5
3.3.1 Mode Clair - Vue principale . . . 5
3.3.2 Mode Sombre - Vue conversation . . . 6
3.3.3 Formulaire de réponse . . . 6

4 Widget Client **7**
4.1 Présentation . . . 7
4.2 Palette de couleurs - Widget . . . 7
4.3 Widget Client - Captures d'écran . . . 7
4.3.1 Page d'accueil avec bouton flottant . . . 7
4.3.2 Bouton flottant . . . 8
4.3.3 Fenêtre de chat ouverte . . . 8
4.3.4 Menu latéral avec historique . . . 9
4.3.5 Mode Sombre du widget . . . 9

5 Démonstration **10**
5.1 Scénarios de test . . . 10
5.1.1 Scénario 1 : Question dans la FAQ . . . 10
5.1.2 Scénario 2 : Question hors FAQ → Escalade Discord . . . 10
5.1.3 Scénario 3 : Upload de document . . . 10

6 Installation et déploiement **11**
6.1 Prérequis . . . 11
6.2 Installation du backend . . . 11
6.3 Installation de l'interface admin . . . 11
6.4 Installation du widget client . . . 11
6.5 Intégration du widget (3 lignes) . . . 11

7 API Endpoints **12**

8 Conclusion du MVP **13**
8.1 Correspondance avec le cahier des charges . . . 13
8.2 Critères de validation . . . 13
8.3 Prochaines étapes du stage . . . 13

Annexes **14**

Introduction

Contexte du projet

Dans le cadre de mon Master 2 en Informatique – Génie Logiciel à l'Université Iba Der Thiam de Thiès (année académique 2025-2026), j'ai développé une plateforme de **Support Client IA multi-applications**. Ce projet répond à un cahier des charges précis pour la mise en place d'un MVP (Minimum Viable Product).

Objectif du MVP

D'après le cahier des charges, l'objectif est de :

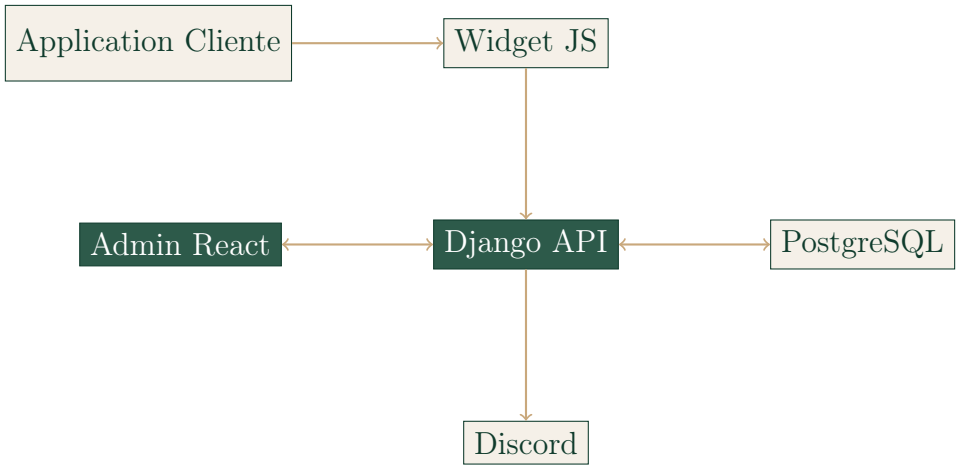
- Intégrer un chatbot dans une application cliente via un widget JavaScript
- Répondre automatiquement via une base de connaissance
- Escalader vers un humain (Discord)
- Permettre à un agent de répondre depuis une application centrale

Technologies utilisées

Backend	Django + Django REST Framework
Base de données	PostgreSQL
Frontend Admin	React + Vite + Axios
Widget Client	React (embarquable)
Notifications	Discord Webhook
OCR	Tesseract pour les images

Architecture de la solution

Architecture globale



Composants principaux

- **Backend Django :** Gère les tenants, conversations, messages et base de connaissance
- **Widget Client :** Interface de chat embarquable en 3 lignes de code
- **Interface Admin :** Permet aux agents de répondre aux conversations escaladées
- **Discord Webhook :** Notifie les agents en cas d'échec du bot

Interface Admin

Présentation

L'interface admin permet aux agents de support de :

- Visualiser toutes les conversations
- Voir les messages en attente (badge "Escaladée")
- Répondre aux clients
- Basculer entre mode clair et sombre

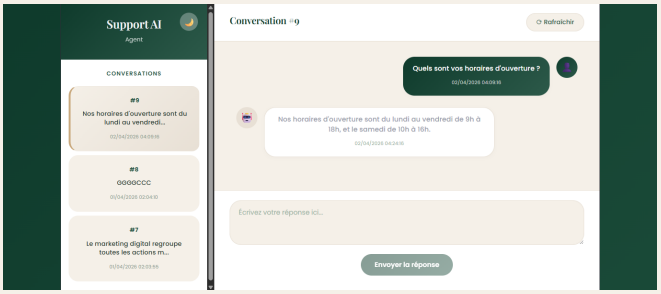
Palette de couleurs

Nom	Code HEX	Utilisation
Vert profond	#0F3B2C	Fonds principaux, gradients
Vert menthe	#2D5A4A	Accents, boutons, titres
Or doux	#C9A87C	Bordures actives, éléments interactifs
Beige clair	#F5F0E8	Fond des messages, arrière-plans
Blanc cassé	#E8E0D5	Cartes, conteneurs
Rouge corail	#E07A5F	Badge "Escaladée"

Interface Admin - Captures d'écran

3.3.1 Mode Clair - Vue principale

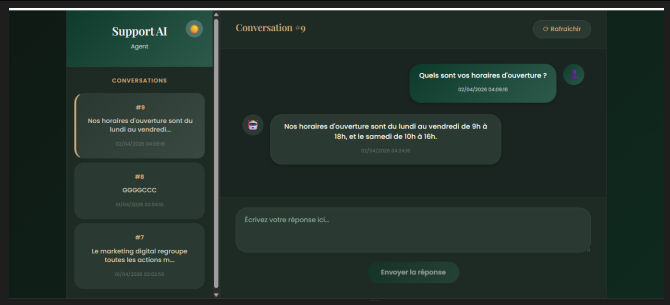
Interface Admin - Mode Clair



Visualisation des conversations avec badge "Escaladée" en rouge corail

3.3.2 Mode Sombre - Vue conversation

Interface Admin - Mode Sombre



Fond vert

foncé, messages gris foncé, accents or doux

3.3.3 Formulaire de réponse

Formulaire de réponse agent

`admin_reponse.png`

L'agent peut

répondre directement depuis l'interface

Widget Client

Présentation

Le widget client est une interface de chat embarquable que les applications tierces peuvent intégrer en seulement 3 lignes de code.

Palette de couleurs - Widget

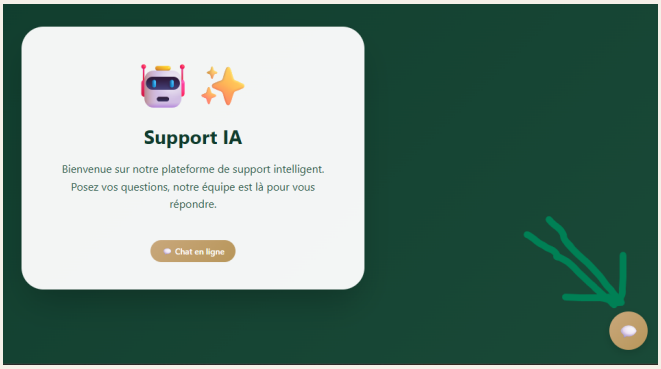
Mode	Élément	Couleur
4*Clair	Fond	#F7F7F8
	Cartes	#FFFFFF
	Messages user	Dégradé #C9A87C → #B8965A
	Messages bot	#FFFFFF
4*Sombre	Fond	#1E1E1E
	Cartes	#2A2A2A
	Messages user	Dégradé #C9A87C → #B8965A
	Messages bot	#2A2A2A

Widget Client - Captures d'écran

4.3.1 Page d'accueil avec bouton flottant

4.3.2 Bouton flottant

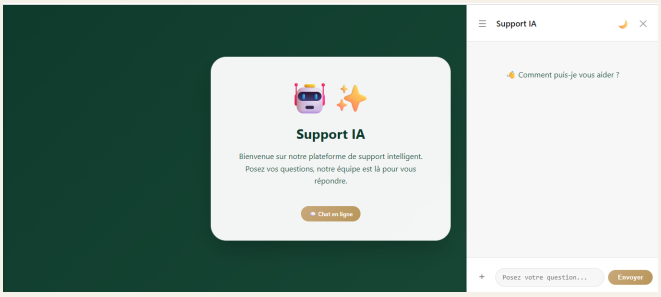
Bouton flottant



Bouton flottant de couleur or doux (#C9A87C)

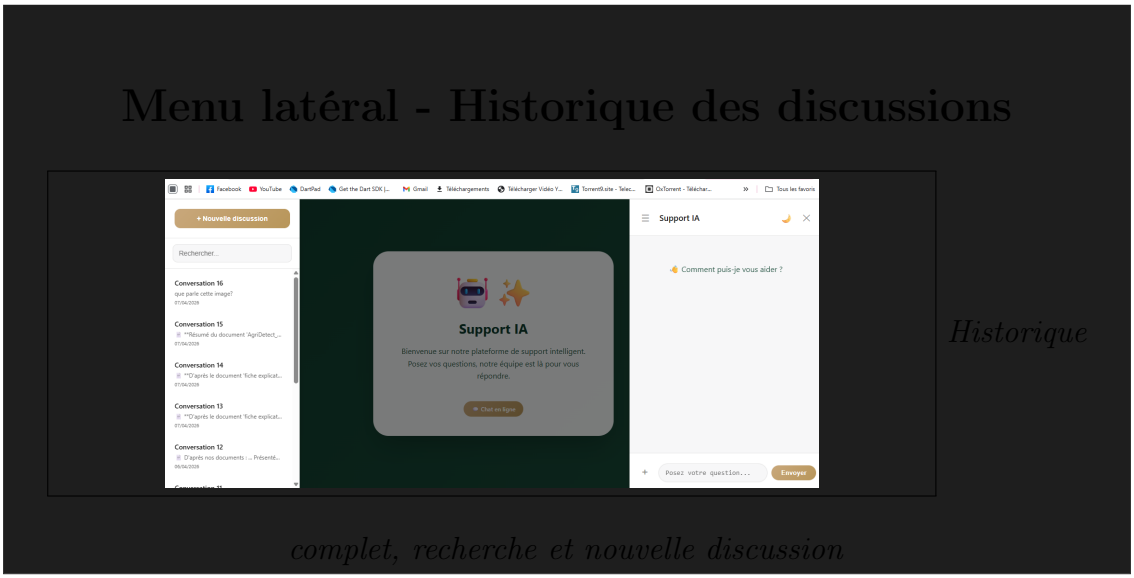
4.3.3 Fenêtre de chat ouverte

Fenêtre de chat latérale



Fenêtre latérale

4.3.4 Menu latéral avec historique



4.3.5 Mode Sombre du widget

Démonstration

Scénarios de test

5.1.1 Scénario 1 : Question dans la FAQ

Client : "Qu'est-ce que le marketing digital ?"

Bot : "Le marketing digital regroupe toutes les actions marketing réalisées sur les canaux numériques : réseaux sociaux, SEO, email marketing, publicité en ligne, etc."

Réponse directe du bot

5.1.2 Scénario 2 : Question hors FAQ → Escalade Discord

Client : "Quel est votre numéro de téléphone ?"

Bot : (aucune réponse affichée)

Discord : Notification reçue avec ID conversation

Agent : Répond via l'admin

Client : Reçoit la réponse de l'agent

Escalade fonctionnelle, client ne voit pas la différence

5.1.3 Scénario 3 : Upload de document

Client : Upload d'un PDF

Système : Extraction du texte complet (PyPDF2 + OCR)

Client : "Que parle ce document ?"

Bot : Résumé avec titre, auteur, contexte et extrait

Document analysé et réponses intelligentes

Installation et déploiement

Prérequis

- Python 3.10+
- PostgreSQL
- Node.js (pour React)
- Git

Installation du backend

```
git clone https://github.com/faLucas17/chatbot-support-platform.git
cd support_platform
python -m venv venv
source venv/bin/activate  # Sur Windows: venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Installation de l'interface admin

```
cd admin
npm install
npm run dev
```

Installation du widget client

```
cd widget
npm install
npm run build
python -m http.server 8080
```

Intégration du widget (3 lignes)

```
<script src="https://unpkg.com/react@18.2.0/umd/react.development.js"></scr
<script src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.development.j
<script src="dist/widget.umd.cjs"></script>
<script>
    ChatWidget.init({
        apiKey: "VOTRE_API_KEY",
        appName: "Nom de votre application",
        theme: "light"
    });
</script>
```

API Endpoints

Méthode	Endpoint	Description
POST	/api/message/	Envoyer un message
GET	/api/conversation/<id>/	Récupérer historique
POST	/api/agent/reply/	Répondre (agent)
POST	/api/documents/upload/	Upload document
GET	/api/documents/	Liste documents
DELETE	/api/documents/<id>/	Supprimer document
GET	/admin/conversations/	Liste conversations

Conclusion du MVP

Correspondance avec le cahier des charges

Exigence du cahier des charges	Statut
Widget JavaScript intégrable	
Chatbot répond via base de connaissance	
Détection d'échec du bot	
Escalade Discord	
Interface admin pour agent	
Polling pour la communication	

Critères de validation

Critère	Statut
Widget fonctionne sur app externe	
Bot répond correctement à une FAQ	
Escalade déclenche Discord	
Agent peut répondre depuis l'admin	
Client reçoit la réponse	

Prochaines étapes du stage

- Intégration de la vraie API Claude
- Support de plus de formats de documents (DOCX, XLSX)
- Authentification multi-utilisateurs
- WebSocket pour le temps réel

Fait à Thiès, le Avril 2026
Fatou FALL

Annexes

Annexe 1 : Structure du projet

```
support_platform/
        manage.py
        requirements.txt
        .env
        chatbot/
            models.py
            views.py
            ai_engine.py
            urls.py
        admin/                      # Interface admin React
            src/
                App.jsx
                App.css
                components/
            package.json
        widget/                     # Widget client
            simple-widget.js
            test.html
            dist/
```

Annexe 2 : Variables d'environnement (.env)

```
ANTHROPIC_API_KEY=sk-...
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
```

django : `pythonmanage.pyrunserver`

react serveur : `npmrundev`

react : `python-mhttp.server8080`

Annexe 3 : Liens utiles

- Dépôt GitHub : https://github.com/faLucas17/chatbot-support-platform
- Documentation Django : https://docs.djangoproject.com
- React : https://react.dev
- admin : http://localhost:5173/
- client : http://localhost:8080/test.html